

MÓDULO: Derivada suavizada por regresión local (Módulo 5)

Nombre: Hector Alexander Perez Natareno

ARQUITECTURA: ARM64

SISTEMA OPERATIVO: GNU/Linux (Raspberry Pi OS / QEMU Emulated Environment)

1. INTRODUCCIÓN Y RESUMEN EJECUTIVO

1.1 Introducción

En el ámbito del análisis de señales y procesamiento de series temporales, la obtención de la tasa de cambio (derivada) es una operación fundamental para identificar tendencias, aceleraciones o anomalías en un sistema. Sin embargo, los métodos de diferenciación numérica tradicionales (como las diferencias finitas hacia adelante o hacia atrás) presentan una alta vulnerabilidad ante el ruido aleatorio de alta frecuencia presente en conjuntos de datos reales. Una fluctuación mínima en una sola muestra se traduce en un pico artificial en la derivada.

Para mitigar este fenómeno, este software implementa un algoritmo de Derivada Suavizada mediante Regresión Lineal Local desarrollado íntegramente en lenguaje ensamblador ARM64 (AArch64). Este módulo procesa de manera eficiente archivos de datos tabulares, aislando ventanas de análisis para ajustar modelos de mínimos cuadrados en tiempo de ejecución, garantizando así un filtrado intrínseco del ruido y una medición precisa de la tendencia real del sistema.

1.2 Resumen Ejecutivo

El programa está diseñado como una utilidad de bajo nivel orientada al rendimiento en entornos Unix/Linux empotrados (como la plataforma Raspberry Pi). Sus características principales incluyen:

- **Entrada:** Interfaz por línea de comandos basada en el estándar POSIX que recibe la ruta de un archivo de datos, el índice de la columna objetivo y los límites de filas (rango) a evaluar.
- **Procesamiento:** Análisis a través de una ventana deslizante de tamaño fijo ($N = 5$) donde se calcula la pendiente local mediante fórmulas matemáticas optimizadas para aritmética entera de 64 bits.
- **Salida:** Generación automática de un archivo de texto estructurado (resultado_derivada.txt) que reporta las métricas calculadas y el estado de la operación (STATUS=OK o STATUS=ERROR).
- **Eficiencia:** Consumo directo de datos desde el Stack (pila) asignado dinámicamente, minimizando las llamadas al sistema (syscalls) de gestión de memoria y garantizando un tiempo de ejecución determinista.

2. EXPLICACIÓN DETALLADA DEL ALGORITMO

2.1 Fundamento Matemático: Mínimos Cuadrados

El algoritmo sustituye la diferenciación simple por una Regresión Lineal por Mínimos Cuadrados aplicada iterativamente sobre un conjunto de ventanas móviles de tamaño $N = 5$. La pendiente de la recta de mejor ajuste para un conjunto de puntos (X_i, Y_i) se calcula mediante la ecuación:

$$\text{LOCAL_SLOPE_X100} = (\text{LOCAL_NUMERADOR} * 100) / \text{LOCAL_DENOMINADOR}$$

$$\text{LOCAL_NUMERADOR} = (5 * \text{suma}(X_i * Y_i)) - (\text{suma}(X_i) * \text{suma}(Y_i))$$

$$\text{LOCAL_DENOMINADOR} = (5 * \text{suma}(X_i * X_i)) - (\text{suma}(X_i) * \text{suma}(X_i))$$

Donde:

- Y_i son los valores numéricos de las muestras extraídas de la serie temporal.
- X_i representa la variable independiente temporal.

2.2 Optimización Estática del Denominador y Coeficientes

Un cálculo de regresión lineal en tiempo real suele requerir operaciones de punto flotante costosas o divisiones dinámicas complejas. No obstante, dado que el tamaño de la ventana está definido como una constante ($N = 5$) y los intervalos temporales locales son uniformes, el algoritmo redefine el eje del tiempo de forma relativa para cada ventana, fijando la variable independiente en el conjunto cerrado: $X_i \in \{0, 1, 2, 3, 4\}$.

Al sustituir estos valores fijos en el denominador original se obtiene:

$$\text{Denominador} = (5 * 30) - (10)^2 = 150 - 100 = 50$$

De igual forma, al sustituir las constantes en el numerador, la ecuación de la pendiente local se simplifica a:

$$\text{Numerador} = 5 (X_i * Y_i) - 10 * Y_i$$

2.3 Escalado de Precisión Entera y Ventana Deslizante

Para conservar la precisión decimal trabajando exclusivamente con la unidad aritmética lógica de enteros (ALU) y la instrucción de división entera `sdiv`, el algoritmo escala el numerador por un factor de 100 antes de efectuar la división. La fórmula final ejecutada a bajo nivel es:

$$\text{LOCAL_SLOPE_X100} = (\text{LOCAL_NUMERADOR} * 100) / \text{LOCAL_DENOMINADOR}$$

El algoritmo inicializa un registro de persistencia en cero para almacenar la pendiente máxima absoluta. El programa desliza la ventana paso a paso a lo largo de la serie temporal. El límite total de iteraciones permitidas para el bucle externo viene dado por la relación:

$$\text{Ventanas Totales} = N - 4$$

En cada iteración, se evalúa la pendiente de la ventana actual, se extrae su valor absoluto y, mediante una estructura de comparación selectiva, se actualiza el registro histórico si la nueva pendiente supera al máximo previo.

3. MAPEO Y ARQUITECTURA DE REGISTROS

El software saca el máximo provecho de la arquitectura de 64 bits de ARM64 utilizando registros de tamaño completo X para punteros, direccionamiento y operaciones matemáticas de gran precisión.

3.1 Registros Temporales y Volátiles, Modificables libremente

- **X0:** Registro de entrada/salida primario. Recibe los punteros a cadenas para conversión de texto, devuelve los enteros procesados por las subrutinas y actúa como el acumulador principal de la operación del numerador.
- **X1:** Registro de propósito general. Se emplea para almacenar multiplicadores inmediatos (como los factores 5, 10, 50 y 100 de la fórmula) y como puntero de rastreo dinámico en la construcción de strings.
- **X2:** Almacena inicialmente la dirección de memoria de la ruta del archivo (argv[1]). Posteriormente, actúa como el receptor del conteo de elementos leídos por la subrutina de la pila.
- **X4: Puntero de Trabajo Indexado.** Rastrea la dirección de memoria exacta del elemento Y_i que se está leyendo dentro del bucle interno de la ventana.
- **X5: Acumulador Lineal.** Almacena de manera incremental el valor de la sumatoria simple de muestras Y_i.
- **X6: Acumulador de Producto.** Almacena el resultado incremental de la sumatoria ponderada por tiempo X_i Y_i.
- **X7: Contador del Bucle Local.** Controla el índice interno de la ventana activa (valores del 0 al 4). Representa físicamente el valor de la variable independiente X_i.
- **X8: Registro de Control.** En el cuerpo del programa controla el índice de la ventana deslizante actual. Al finalizar, almacena el número de la llamada al sistema (#93 para la salida limpia del programa).
- **X9: Registro de Condición de Parada.** Guarda el valor límite de ventanas físicas que se pueden procesar N - 4.
- **X10: Calculador de Desplazamientos (Offset).** Almacena el cálculo temporal en bytes necesario para reposicionar el puntero de la pila al cambiar de ventana Índice x 16 bytes.

3.2 Registros Preservados “Conservan estado”

- **X11: Preservación de Metadatos.** Almacena el índice entero de la columna analizada, manteniéndolo a salvo de las destrucciones de registros de las subrutinas auxiliares.
- **X19: Dirección Base Inferior.** Almacena el puntero inicial del bloque de datos devuelto por la subrutina de carga.
- **X20: Dirección Base Superior.** Almacena el puntero al final del bloque de datos en memoria. Es el punto de partida desde donde el algoritmo lee los datos.
- **X21: Contador de Registros Totales (N).** Guarda la cantidad exacta de registros válidos leídos de la columna. Se usa para validar el tamaño mínimo del rango.
- **X22: Registro Ancla del Stack.** Guarda la dirección original del Stack Pointer (SP) antes de la inyección de datos del archivo CSV. Es crítico para la estabilidad del sistema operativo.
- **X23: Registro de Récord de Pendiente.** Almacena el valor absoluto de la pendiente más alta calculada a lo largo de toda la serie de datos.
- **X24: Línea Inicial.** Preserva de forma intacta el identificador de la fila inicial del rango.
- **X25: Línea Final.** Preserva de forma intacta el identificador de la fila final del rango.

4. ARQUITECTURA DE DATOS EN MEMORIA (SECCIONES)

El binario organiza la memoria en tres secciones de acuerdo con el estándar de formato ELF (Executable and Linkable Format): `.data`, `.bss`, y `.text`. Esta segmentación optimiza los accesos a memoria y proporciona protección a nivel de páginas del sistema operativo (lectura, escritura y ejecución).

4.1 Sección `.data` (Datos Inicializados)

Esta sección aloja las constantes alfanuméricas fijas del programa. Todas las cadenas se definen mediante la directiva `.asciz`, lo que garantiza que el compilador inserte automáticamente un carácter nulo terminador (`\0` o byte cero) al final de cada string. Se aplica un alineamiento estricto a frontera de 8 bytes (`.align 3`) para optimizar el bus de datos del procesador.

- `archivo_salida`: Almacena "resultado_derivada.txt".
- Etiquetas de formateo del reporte: `etq_calc`, `etq_column`, `etq_win_start`, `etq_win_end`, `etq_count`, `etq_win_size`, `etq_max_slope`, `etq_status_ok`.
- `etq_status_err`: Cadena preformateada que contiene el reporte estructurado completo en caso de error por datos insuficientes.

4.2 Sección .bss (Datos No Inicializados)

Reserva memoria estática que se inicializará en cero al cargar el programa. Su uso está destinado a buffers temporales de escritura:

- `num_buffer`: Espacio reservado de 32 bytes (`.skip 32`). Es la memoria intermedia donde la función `utils_itoa` escribe los caracteres ASCII resultantes de convertir un número binario.
- `buffer_salida`: Espacio reservado de 512 bytes (`.skip 512`). Es el área de memoria donde el programa va construyendo secuencialmente, línea por línea, el archivo de texto final mediante la concatenación de las etiquetas y valores numéricos.

4.3 Organización de los Datos Dinámicos en el Stack

Una de las genialidades de diseño del programa es el uso del Stack (pila) del sistema como estructura de datos dinámica para almacenar los números leídos del archivo CSV.

Cuando la subrutina externa `utils_read_column_to_stack` procesa el archivo, cada número entero extraído se empuja (`push`) a la pila. En la arquitectura ARM64, el stack es descendente (crece hacia direcciones de memoria más bajas). Cada registro de datos inyectado ocupa exactamente **16 bytes** de espacio contiguo (8 bytes para el valor numérico y 8 bytes para alineación/control).

Debido a esta estructura física de la pila, la distribución de los datos en memoria presenta un orden inverso respecto a la lectura del archivo:

- La dirección de memoria más alta (X20) contiene el primer dato leído del archivo (el más antiguo cronológicamente).
- La dirección de memoria más baja (X19) contiene el último dato leído del archivo (el más reciente).

5. DISEÑO DE FLUJO LINEAL Y ABSTRACCIÓN DE STRINGS

5.1 Flujo Lineal del Programa

El programa sigue una ruta de ejecución lineal y determinista que garantiza la predictibilidad del software. Las etapas se dividen en:

1. **Validación de Argumentos**: Compara `argc` contra el valor mínimo requerido. Si falla, salta directamente a la rutina de terminación por error.
2. **Conversión de Parámetros**: Invoca a `atoi_argv` para traducir los argumentos de texto a enteros legibles por el programa.
3. **Carga en Memoria**: Llama a la rutina externa para volcar la columna del archivo directo en la pila del sistema.

4. **Validación de Datos:** Compara si la cantidad de muestras leídas es menor a 5. Si es así, bifurca hacia la construcción del reporte de error.
5. **Procesamiento Matemático:** Ejecuta los ciclos anidados de cálculo de pendiente de las ventanas deslizantes.
6. **Construcción del Reporte:** Concatena las cadenas y resultados numéricos en el buffer intermedio.
7. **Escritura y Cierre:** Genera el archivo físico en el almacenamiento y restaura el estado del microprocesador.

5.2 Abstracción de Strings (Concatenación Segura)

El manejo de strings en ensamblador requiere una gestión manual exhaustiva de los punteros de memoria para evitar corrupciones o desbordamientos de búfer (buffer overflows). El programa implementa dos abstracciones de alto nivel denominadas `append_string` y `append_number`.

Subrutina `append_string`

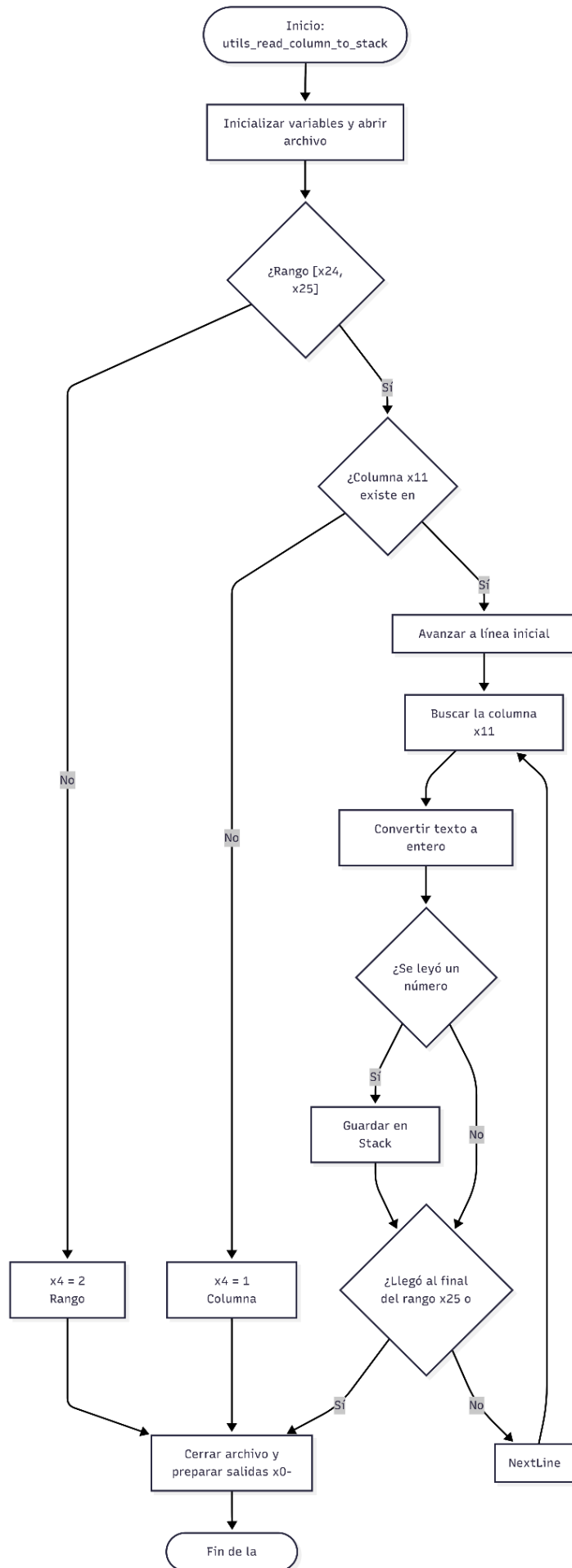
Esta rutina actúa de manera análoga a la función estándar `strcat` de C. Recibe en X0 la dirección de la cadena origen que desea añadir al reporte.

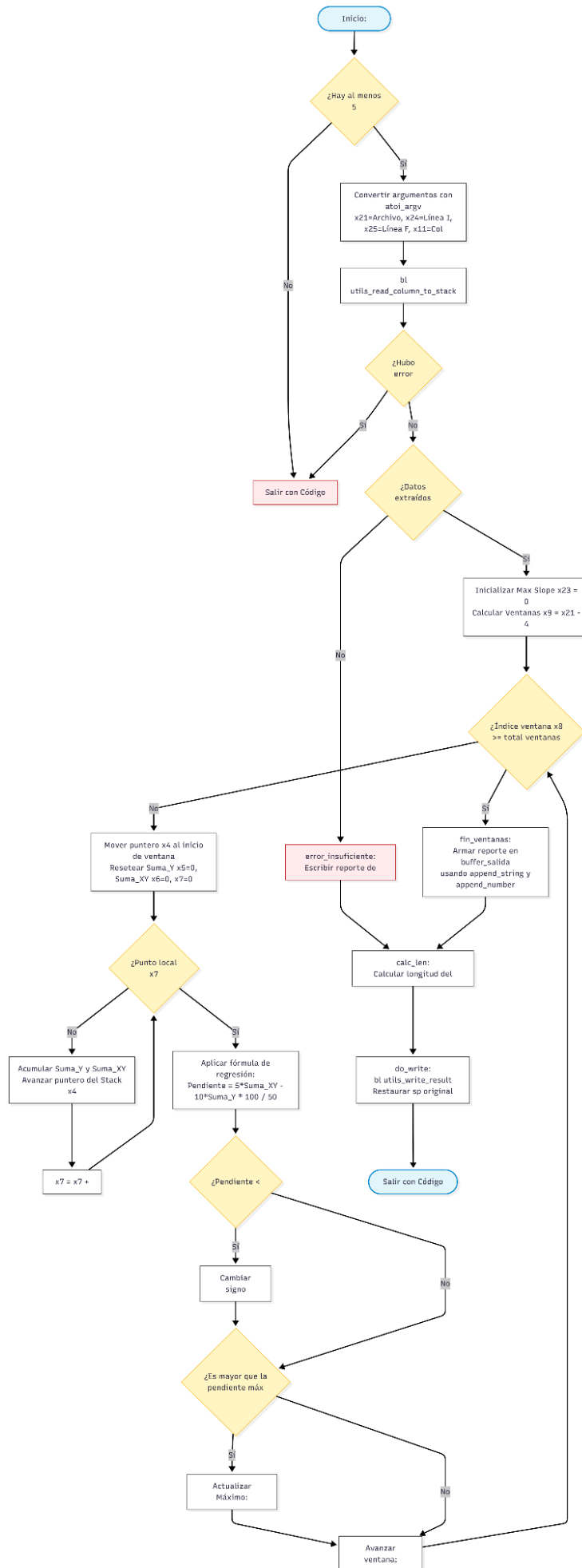
1. Carga la dirección base del `buffer_salida` en X1.
2. Ejecuta un ciclo de búsqueda activa (`find_end`) leyendo byte a byte (`ldrb w2, [x1]`) e incrementando el puntero en uno hasta localizar el carácter nulo (`\0`). Esto asegura que no se sobrescriba el texto previamente guardado.
3. Al posicionarse sobre el fin real de la cadena, la rutina pasa a la fase `copy_str`. Lee un byte de la cadena de origen, lo almacena en la posición de la cadena de destino y avanza ambos punteros simultáneamente (`ldrb w2, [x0], #1` y `strb w2, [x1], #1`).
4. El ciclo se detiene de forma automática cuando el byte copiado es el carácter nulo (`\0`), consolidando la unión segura de los textos.

Subrutina `append_number`

Esta rutina elimina la complejidad de concatenar datos numéricos binarios directos en un flujo de texto.

1. Guarda el estado del procesador en el stack (`stp x29, x30, [sp, #-16]!`) para asegurar el retorno correcto tras las llamadas anidadas.
2. Invoca a `utils_itoa`, la cual toma el valor entero de X0, realiza internamente divisiones sucesivas entre 10 para aislar los dígitos y escribe los caracteres ASCII correspondientes dentro de `num_buffer`.
3. Inmediatamente después, carga la dirección de `num_buffer` en X0 e invoca a `append_string` para acoplar de forma inmediata el número ya convertido al reporte general.





6. INSTRUCCIONES DE CONTROL Y OPERACIONES MECÁNICAS DE BAJO NIVEL

La eficiencia óptima reside en el uso selectivo de instrucciones nativas de la arquitectura ARM64, evitando dependencias externas y aprovechando la ejecución directa en el pipeline de la CPU.

6.1 Operaciones Aritméticas Críticas

- `mul x0, x0, x7`: Multiplicación de 64 bits. Multiplica el valor del dato por el índice local de tiempo. Es fundamental para obtener el producto intermedio de la sumatoria $X_i Y_i$.
- `sdiv x0, x0, x1`: División entera con signo de 64 bits. Divide el numerador escalado alojado en X0 entre la constante 50 guardada en X1. Esta instrucción se ejecuta de forma directa en hardware en los núcleos ARM64, ofreciendo un rendimiento muy superior al de algoritmos basados en restas sucesivas.
- `neg x0, x0`: Negación aritmética con signo. Invierte el signo del valor numérico contenido en el registro ($X0 = -X0$). Se utiliza específicamente para el cálculo del valor absoluto de la pendiente cuando la evaluación de banderas detecta un número menor que cero.

6.2 Instrucciones de Control de Flujo e Indexación

- `cbz w2, do_write` (Compare and Branch if Zero): Es una instrucción altamente optimizada que fusiona la comparación y el salto en un único ciclo de reloj. Evalúa si el registro de 32 bits W2 es igual a cero; si la condición se cumple, salta instantáneamente a la etiqueta seleccionada. Se utiliza para identificar el final de las cadenas de texto.
- `cbnz w2, copy_str` (Compare and Branch if Non-Zero): Complementaria a la anterior, salta si el carácter evaluado no es cero, manteniendo activo el bucle de clonación de strings de forma eficiente.
- `ldrb w2, [x0], #1` (Load Register Byte con Post-Indexado): Extrae un único byte (8 bits) desde la dirección de memoria apuntada por X0 y lo deposita en el registro de tamaño reducido W2. La operación mecánica de post-indexado (#1) incrementa automáticamente el valor del puntero X0 en una unidad justo después de realizar la lectura, eliminando la necesidad de programar una instrucción adicional `add x0, x0, #1`.

6.3 Mecánica de Restauración Absoluta del Stack

La gestión segura de la pila es vital en el desarrollo de software de bajo nivel. Cuando la biblioteca de lectura inyecta datos en el stack, el puntero de la pila (SP) se desplaza dinámicamente una distancia indeterminada que depende del tamaño del archivo CSV. Si el programa intentara retornar o finalizar sin corregir esto, el puntero de la pila apuntaría a una dirección incorrecta, corrompiendo el mapa de memoria del proceso.

Para solucionar esto, el código emplea un mecanismo de anclaje estático:

1. Al iniciar el programa, la instrucción `mov x22, x3` resguarda permanentemente el valor original del puntero en el registro no volátil X22.

2. Una vez completadas todas las tareas aritméticas y de escritura de archivos, la instrucción `mov sp, x22` restaura de golpe el estado original del Stack Pointer. Esta operación mecánica invalida instantáneamente todo el espacio temporal de datos consumido por el CSV, liberando la memoria de forma limpia y segura sin incurrir en costos de recolección de basura o desasignaciones iterativas de memoria.
3. Finalmente, el programa invoca el apagado seguro devolviendo el control al kernel de Linux mediante el código de salida de interrupción por software:

```
mov x0, #0 // Código de retorno 0 (Éxito)
```

```
mov x8, #93 // Identificador del Syscall 'exit' en ARM64 Linux
```

```
svc #0 // Supervisor Call (Bifurcación al Kernel)
```

7. APÉNDICE: ANÁLISIS DE CASOS DE BORDE Y TOLERANCIA A FALLOS

El diseño del código maneja los escenarios críticos de error y las limitaciones físicas de la arquitectura.

7.1 Manejo del Límite Estricto de Muestras ($N < 5$)

El algoritmo de regresión por mínimos cuadrados local está matemáticamente acotado para un tamaño de muestra invariable de 5 puntos. Si el usuario define por comandos un rango de filas que contiene 4 o menos muestras (por ejemplo, analizar de la línea 10 a la 12), el denominador de la regresión lineal cambiaría y la ventana de procesamiento fija fallaría.

El programa gestiona este caso de borde antes de entrar a la sección de cómputo:

```
cmp x21, #5
```

```
blt error_insuficiente
```

Si la cantidad total de datos reales en la pila (X21) es inferior a 5, el salto condicional `blt` interrumpe el flujo normal y desvía el código hacia la etiqueta `error_insuficiente`. Esta limpia el buffer e imprime directamente la estructura preformateada de error, impidiendo que el procesador intente acceder de forma inválida a posiciones vacías de la pila y asegurando una salida controlada con código informativo.

Demostración de depuración con GDB

```
B+> 73 stp x29, x30, [sp, #-16]! // Guardar frame pointer (x29) y link register (x30)
74 mov x29, sp // Fijar frame pointer al stack actual
75
76 mov x28, sp // x28 = límite superior de datos (dirección más alta)
77 add x27, x28, #16 // x27 = posición segura para restaurar el stack original
78
79 mov x22, #0 // x22 = contador de números extraídos
80 mov x4, #0 // x4 = código de estado de salida (0 = OK)
81
82 // Abrir archivo y resetear buf_pos / buf_len
83 bl utils_open_file // Llamada para abrir lecturas.csv
84
85 cmp x24, #1
86 blt utils_invalid_range // Si línea inicial < 1 - error de rango
87
88 cmp x24, x25
89 bgt utils_invalid_range // Si línea inicial > línea final - error de rango
90
91 mov x12, #1 // x12 = columnas vistas hasta ahora (arranca en 1)
92
93 utils_count_header:
94 bl utils_next_char // w23 = siguiente carácter del stream
95 cmp w23, '$'
96 beq utils_invalid_column // EOF antes de terminar el encabezado - archivo mal formado
97 cmp w23, #10 // Comparar con salto de línea (\n) - fin del encabezado
98 beq utils_check_col_exists // Saltar a validar si la columna pedida existe
99 cmp w23, ',' // Comparar con coma separadora de columnas
100 bne utils_count_header // Si no es coma, seguir leyendo el encabezado
101 add x12, x12, #1 // Una coma más = una columna más
102 b utils_count_header // Continuar recorriendo el encabezado
103
104 utils_check_col_exists:
105 cmp x11, x12 // ¿La columna pedida (x11) es mayor que las columnas del archivo (x12)?
106 bgt utils_invalid_column // Si x11 > x12 - columna fuera de rango - error
107
108 mov x26, #0 // x26 = línea de datos actual (contador interno, empieza en 0)
109
110 utils_skip_before_start:
```

remote Thread 3568.3568 (src) In: utils_read_column_to_stack L73 PC: 0x4002a8

(gdb) break _start
Breakpoint 1 at 0x4000b0: file modulo_5_derivada_local.s, line 46.
(gdb) break utils_read_column_to_stack
Breakpoint 2 at 0x4002a8: file utils.s, line 73.
(gdb) break loop_ventanas
Breakpoint 3 at 0x400118: file modulo_5_derivada_local.s, line 85.
(gdb) break fin_ventanas
Breakpoint 4 at 0x4001a4: file modulo_5_derivada_local.s, line 136.
(gdb) continue
Continuing.

Breakpoint 2, utils_read_column_to_stack () at utils.s:73
(gdb) info registers x0
x0 0x400007ffdb3 70368752565683
(gdb) x/5gx \$sp
0x400007ffac0: 0x0000000000000005 0x0000400007ffdb3c
0x400007ffad0: 0x0000400007ffdb9f 0x0000400007ffdbac
0x400007ffae0: 0x0000400007ffdaef 0x0000400007ffdbae
(gdb)

```
89 bgt utils_invalid_range // Si línea inicial > línea final - error de rango
90
91 mov x12, #1 // x12 = columnas vistas hasta ahora (arranca en 1)
92
93 utils_count_header:
94 bl utils_next_char // w23 = siguiente carácter del stream
95 cmp w23, '$'
96 beq utils_invalid_column // EOF antes de terminar el encabezado - archivo mal formado
97 cmp w23, #10 // Comparar con salto de línea (\n) - fin del encabezado
98 beq utils_check_col_exists // Saltar a validar si la columna pedida existe
99 cmp w23, ',' // Comparar con coma separadora de columnas
100 bne utils_count_header // Si no es coma, seguir leyendo el encabezado
101 add x12, x12, #1 // Una coma más = una columna más
102 b utils_count_header // Continuar recorriendo el encabezado
103
104 utils_check_col_exists:
105 cmp x11, x12 // ¿La columna pedida (x11) es mayor que las columnas del archivo (x12)?
106 bgt utils_invalid_column // Si x11 > x12 - columna fuera de rango - error
107
108 mov x26, #0 // x26 = línea de datos actual (contador interno, empieza en 0)
109
110 utils_skip_before_start:
```

remote Thread 3568.3568 (src) In: utils_read_column_to_stack L73 PC: 0x4002a8

Breakpoint 4 at 0x4001a4: file modulo_5_derivada_local.s, line 136.
(gdb) continue
Continuing.

Breakpoint 2, utils_read_column_to_stack () at utils.s:73
(gdb) info registers x0
x0 0x400007ffdb3 70368752565683
(gdb) x/5gx \$sp
0x400007ffac0: 0x0000000000000005 0x0000400007ffdb3c
0x400007ffad0: 0x0000400007ffdb9f 0x0000400007ffdbac
0x400007ffae0: 0x0000400007ffdaef 0x0000400007ffdbae
(gdb) print/x \$x11
\$1 = 0x2
(gdb) print/x \$x24
\$2 = 0x1
(gdb) print/x \$x25
\$3 = 0x1e
(gdb) print \$x21
\$4 = 70368752565663
(gdb)

```

79  mov x23, #0          // valor absoluto maximo en contrado de momento
80  mov x8, #0           // indice de la ventana actual
81
82  sub x9, x21, #4       // cantidad de ventanas posibles. n - 4
83
84  loop_ventanas:
85      cmp x8, x9        // ver si se procesaron ya todas las ventanas posibles
86      bge fin_ventanas  // si si, se termina el ciclo principal
87
88      sub x4, x20, #16   // direccion del dato mas antiguo del rango leido
89      mov x10, #16      // tamaño de bytes de cada dato almacenado
90      mul x10, x8, x10   // indice de ventana * 16
91      sub x4, x4, x10    // puntero al primer dato
92
93      mov x5, #0         // suma(Y_i) - acumulador de los valores de y de la ventana
94      mov x6, #0         // suma(X_i * Y_i) - acumulador de los productos x*y
95      mov x7, #0         // X local, posicion dentro de la ventana: 0,1,2,3,4,
96
97  loop_5_puntos:
98      cmp x7, #5        // ver si ya se procesaron los 5 puntos de esta ventana
99      bge fin_5_puntos  // si si, se sale del ciclo
100
101      ldr x0, [x4]       // x0 = Y_i, el valor actual de la ventana
102      add x5, x5, x0     // suma_Y += Y_i
103      mul x0, x0, x7     // X_i * Y_i, X_i es la posicion local de 0 a 4 dentro de la ventana
104      add x6, x6, x0     // suma_XY += X_i * Y_i

```

remote Thread 3568.3568 (src) In: loop_ventanas

L85 PC: 0x400118

\$1 = 0x2

(gdb) print/x \$x24

\$2 = 0x1

(gdb) print/x \$x25

\$3 = 0x1e

(gdb) print \$x21

\$4 = 70368752565663

(gdb) continue

Continuing.

Breakpoint 3, loop_ventanas () at modulo_5_derivada_local.s:85

(gdb) print \$x8

\$5 = 0

(gdb) print \$x9

\$6 = 26

(gdb) print \$x20

\$7 = 70368752564912

(gdb) x/5gx \$x4

```

94      mov x6, #0         // suma(X_i * Y_i) - acumulador de los productos x*y
95      mov x7, #0         // X local, posicion dentro de la ventana: 0,1,2,3,4,
96
97  loop_5_puntos:
98      cmp x7, #5        // ver si ya se procesaron los 5 puntos de esta ventana
99      bge fin_5_puntos  // si si, se sale del ciclo
100
101      ldr x0, [x4]       // x0 = Y_i, el valor actual de la ventana
102      add x5, x5, x0     // suma_Y += Y_i
103      mul x0, x0, x7     // X_i * Y_i, X_i es la posicion local de 0 a 4 dentro de la ventana
104      add x6, x6, x0     // suma_XY += X_i * Y_i
105
106      sub x4, x4, #16    // se avanza el puntero al siguiente dato
107      add x7, x7, #1     // se incrementa X local
108      b loop_5_puntos   // se repite hasta que se completen los 5 puntos
109
110  fin_5_puntos:
111      // se aplica la formula LOCAL_SLOPE_X100 = ((5 * suma_XY) - (10 * suma_Y)) * 100 / 50
112      mov x0, #5        // x0 = 5
113      mul x0, x0, x6     // x0 = 5 * suma_XY
114      mov x1, #10       // x1 = 10
115      mul x1, x1, x5     // x1 = 10 * suma_Y
116      sub x0, x0, x1     // x0 = numerador = (5*suma_XY) - (10*suma_Y)
117      mov x1, #100      // x1= 100
118      mul x0, x0, x1     // x0 = numerador * 100
119      mov x1, #50       // x1 = 50
120      sdiv x0, x0, x1    // x0 = LOCAL_SLOPE_X100
121
122      // se calcula el valor absoluto de la pendiente para compararla con el maximo
123      cmp x0, #0        // se ve si es negativa
124      bge slope_positivo // si no, no se hace nada
125      neg x0, x0        // si si, se cambia el signo
126
127  slope_positivo:
128      cmp x0, x23       // se ve si la pendiente supera al maximo actual
129      ble slope_no_es_max // si no, no se actualiza
130      mov x23, x0       // si si, se actualiza con la nueva pendiente
131  slope_no_es_max:
132

```

remote Thread 3568.3568 (src) In: fin_5_puntos

L113 PC: 0x400168

\$6 = 26

(gdb) print \$x20

\$7 = 70368752564912

(gdb) x/5gx \$x4

0x0: Cannot access memory at address 0x0

(gdb) break fin_5_puntos

Breakpoint 5 at 0x400160: file modulo_5_derivada_local.s, line 111.

(gdb) continue

Continuing.

Breakpoint 5, fin_5_puntos () at modulo_5_derivada_local.s:111

(gdb) print \$x5

\$8 = 145

(gdb) print \$x6

\$9 = 285

(gdb) next

(gdb) next

(gdb) print \$x0

\$10 = 1425

(gdb)

```

77 blt error_insuficiente // si no, ir al mensaje de error estructurado
78
79 mov x23, #0 // valor absoluto maximo en contrato de momento
80 mov x8, #0 // indice de la ventana actual
81
82 sub x9, x21, #4 // cantidad de ventanas posibles. n - 4
83
84 loop_ventanas:
85 cmp x8, x9 // ver si se procesaron ya todas las ventanas posibles
86 bge fin_ventanas //si si, se termina el ciclo principal
87
88 sub x4, x20, #16 // direccion del dato mas antiguo del rango leido
89 mov x10, #16 // tamaño de bytes de cada dato almacenado
90 mul x10, x8, x10 // indice de ventana * 16
91 sub x4, x4, x10 // puntero al primer dato
92
93 mov x5, #0 // suma(Y_i) - acumulador de los valores de y de la ventana
94 mov x6, #0 // suma(X_i * Y_i) - acumulador de los productos x*y
95 mov x7, #0 // X local, posicion dentro de la ventana: 0,1,2,3,4,
96
97 loop_5_puntos:
98 cmp x7, #5 // ver si ya se procesaron los 5 puntos de esta ventana
99 bge fin_5_puntos // si si, se sale del ciclo
100
101 ldr x0, [x4] //x0 = Y_i, el valor actual de la ventana
102 add x5, x5, x0 // suma_Y += Y_i
103 mul x0, x0, x7 // X_i * Y_i, X_i es la posicion local de 0 a 4 dentro de la ventana
104 add x6, x6, x0 // suma_XY += X_i * Y_i

```

remote Thread 3568.3568 (src) In: loop_ventanas

L85 PC: 0x400118

Breakpoint 5 at 0x400160: file modulo_5_derivada_local.s, line 111.

(gdb) continue

Continuing.

Breakpoint 5, fin_5_puntos () at modulo_5_derivada_local.s:111

(gdb) print \$x5

\$5 = 145

(gdb) print \$x6

\$6 = 285

(gdb) next

(gdb) next

(gdb) print \$x0

\$10 = 1425

(gdb) continue

Continuing.

Breakpoint 3, loop_ventanas () at modulo_5_derivada_local.s:85

(gdb) print \$x23

\$11 = 50

(gdb) []

WSL: Ubuntu-24.04

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

hapn1@HAPN:/mnt/c/Users/hapn1/OneDrive/modulo5\$ cat resultado_derivada.txt

CALC=LOCAL_DERIVATIVE

COLUMN=2

WINDOW_START=1

WINDOW_END=30

COUNT=30

WINDOW_SIZE=5

MAX_LOCAL_SLOPE_X100=250

STATUS=OK

hapn1@HAPN:/mnt/c/Users/hapn1/OneDrive/modulo5\$